Technical Design Report: AI-Driven Financial Reconciliation Pipeline

1. Executive Summary

This project implements an unsupervised, continuous-learning pipeline designed to automatically reconcile check register entries with bank statement transactions. The system successfully addresses the "cold start" problem inherent in unsupervised clustering by employing a hybrid architecture. It first establishes a deterministic "ground truth" via unique monetary amounts, uses that subset to dynamically learn the bank's temporal processing patterns, and finally applies a pre-trained Large Language Model (Sentence Transformer) to resolve ambiguous edge cases via a weighted confidence algorithm.

2. System Architecture & Methodology

The reconciliation pipeline operates in three distinct phases to maximize both precision and recall without requiring manually labeled training data.

Phase 1: Deterministic Seeding (The "Cold Start" Solution)

In an entirely unsupervised environment, evaluating model accuracy and tuning temporal penalties is highly prone to error. To circumvent this, the system isolates transactions where the monetary amount appears exactly *once* in the Bank Statement and exactly *once* in the Check Register.

Assumption: If a highly specific, non-round monetary amount (e.g., $142.87) is unique to both datasets, the probability that these records represent the same underlying transaction approaches 100%.

By merging these 1-to-1 unique pairs, the system securely resolved the vast majority of the dataset (286 transactions) and created a perfectly accurate, parallel corpus to train the next phase of the pipeline.

Phase 1.5: Temporal Pattern Extraction (Date Lag Distribution)

Instead of hardcoding a static assumption (e.g., "bank transfers take 3 days"), the system dynamically learns the specific clearing behaviors of the synthetic bank.

- Using the 286 perfect matches from Phase 1, the system calculates the date difference (lag) in days between when a check was written and when it cleared the bank.
- It aggregates these lags into a **Probabilistic Distribution**. For example, if a 2-day lag occurs in 45% of the deterministic matches, the base probability of a 2-day lag is recorded as 0.45.
- This distribution is normalized (setting the most frequent lag to 1.0) to serve as the P_{date} penalty weight for the Machine Learning phase.

Phase 2: Semantic Machine Learning (Handling Edge Cases)

For the remaining ambiguous transactions (identical amounts, overlapping dates, differing descriptions), the system employs a pre-trained Sentence Transformer (all-MiniLM-L6-v2) to perform semantic matching.

Text Normalization & Noise Reduction: Before vectorization, the text undergoes strict cleaning. A key observation was the presence of random transaction identifiers (e.g., #9928) appended to descriptions. A Regular Expression (Regex) was implemented to strip these numeric artifacts,

ensuring the embeddings captured the pure semantic intent of the vendor names rather than overfitting to noise.

3. The Confidence Score Algorithm

To evaluate potential matches in Phase 2, the system calculates an aggregate Confidence Score for every possible combination of unmatched bank and check transactions. The score is a weighted sum of three distinct similarity metrics:

Confidence = (α * S_text) + (β * P_date) + (γ * S_amount)

Where:

1. **S_{text} (Semantic Similarity - Weight: α = 0.6):** The Cosine Similarity between the vector embeddings of the cleaned descriptions. This is given the highest weight, as vendor names are the strongest indicator of a match when amounts are non-unique.
2. **P_{date} (Temporal Probability - Weight: β = 0.3):** The normalized probability of the date lag, queried directly from the distribution learned in Phase 1.5. If a proposed match has a lag of 10 days, but a 10-day lag was never observed in Phase 1, this score drops to 0.0, heavily penalizing the match.
3. **S_{amount} (Monetary Similarity - Weight: γ = 0.1):** Calculated as a linear decay penalty based on the absolute difference between the amounts. A difference of $0 yields a 1.0, while differences greater than $10 yield a 0.0.

Matches that achieve a final Confidence Score of ≥ 0.5 are accepted and exported to the final reconciliation report.

4. Evaluation Strategy & Data Integrity

Because the assignment utilizes synthetic data without an explicit ground truth key, evaluating the system required careful feature engineering to avoid data leakage.

It was observed that the transaction_id columns shared a deterministic substring pattern (e.g., Bank ID B1234 corresponds to Check ID R1234).

- **Preventing Data Leakage:** The ML model (matcher.py) was strictly isolated from these IDs. The Confidence Score was calculated *exclusively* using the Description, Amount, and Date.
- **Automated Evaluation:** The ID pattern was leveraged exclusively in Phase 3 (evaluator.py) to dynamically generate an Answer Key. This allowed for immediate, programmatic calculation of Precision, Recall, and F1 metrics without the model ever "cheating" by looking at the IDs during inference.

5. Trade-offs and Future Improvements

While the current architecture is highly robust for the provided dataset, several design trade-offs were made:

1. **Pre-trained Embeddings vs. SVD:** The academic paper outlining this workflow relies on TF-IDF and Singular Value Decomposition (SVD). I opted to replace SVD with a modern Sentence Transformer. While SVD must be trained from scratch on historical data (requiring months of historical matches to build a vocabulary), Sentence Transformers possess generalized

semantic understanding out-of-the-box, making them vastly superior for the "cold start" phase of a new reconciliation pipeline.

2. **In-Memory Compute vs. Vector Database:** For this dataset (308 rows), calculating the Cosine Similarity matrix in-memory via scikit-learn is virtually instantaneous. However, if deployed to production processing millions of transactions, calculating $N \times M$ combinations becomes computationally unfeasible. The architecture would need to be upgraded to store the check embeddings in a Vector Database (like Pinecone or Milvus) and query the top-K nearest neighbors for each bank transaction.
3. **Handling 1-to-Many Matches:** The current pipeline operates under a strict 1-to-1 assumption. In reality, a single bank deposit might represent three separate check register entries. Future iterations would require an aggressive combinatorial algorithm (like the Subset Sum problem) to aggregate checks before passing them to the Semantic Matcher.